EXPERIMENT 14

Exploring Pro-Code Extension of a Zoho Creator Application Using Zoho Catalyst and Custom Functions

LOW CODE APPLICATION • LABORATORY REPORT

Student Name	Aravindh Shankar
Roll Number	241401012
Programme / Year	B.Tech Computer Science and Business Systems — III Year
Institution	Rajalakshmi Engineering College
Course	Low Code Application

241401012

Experiment aim

To explain how a low-code application can be extended with custom functions or external services when requirements exceed the visual builder's capabilities.

Application / case context

The Gym Enrollment app already uses Deluge workflows for age and start-date validation and for email confirmation. A pro-code extension could provide more complex integrations, reusable services, or advanced processing. No Catalyst service is shown in the supplied screenshots, so this report presents an extension design and procedure rather than claiming that a Catalyst deployment exists.

Report structure: background, design, procedure, verification, results, conclusion and references. Any feature not confirmed by supplied evidence is identified as a proposed design or procedure.

Student: Aravindh Shankar • Roll No. 241401012 • Rajalakshmi Engineering College

1. Aim, Objectives and Extension Concepts

Low-code platforms cover common form, report, and workflow needs. Pro-code extensions are useful when an application requires specialized algorithms, custom APIs, complex integrations, reusable services, or development/testing controls beyond the visual builder.

Objectives: distinguish Deluge custom functions from an external service; identify when an extension is justified; define an interface between Creator and the service; consider authentication and failure handling; and test the integrated behaviour.

Deluge custom functions are useful for reusable logic within the Zoho environment. Zoho Catalyst can be used to build and host application services using supported runtimes and components, subject to its current product capabilities and plan.

Pro-code extension should solve a real limitation, not be added simply to make the architecture look more advanced. Simple age validation and date checks are understandable inside Creator. Moving them to an external service would introduce another dependency and could make ordinary form submissions less reliable.

An external API should have a documented contract: required input fields, response fields, authentication method, timeout, error codes, and versioning policy. Validate inputs on the server even if the Creator form already validates them, because an endpoint may later be called by another client that does not use the same form.

Operational responsibility increases with a pro-code extension. The team must manage deployment, secrets, logging, monitoring, retries, and service failures. Use a stable request identifier to make retries safe, and make sure the Creator workflow can distinguish a successful response from a timeout or an invalid response.

Keep the Creator app functional when an external service is temporarily unavailable, or define clearly when the operation must fail closed. The correct behaviour depends on business risk. Test service recovery and ensure retries do not duplicate records or send duplicate notifications.

The scope of this experiment is deliberately limited to a lab-scale application. It demonstrates the underlying concept and the verification method; it does not establish production readiness without the additional checks described in this report.

2414n1033

Objectives and scope

This experiment studies how Deluge custom functions and external services can extend Creator when the requirements justify additional software components.

2. Choosing the Extension Boundary

Keep simple validation in Creator where possible: age range, date checks, required fields, and email confirmation are straightforward workflow responsibilities. Moving them to an external service would add latency and failure points without a clear benefit.

Consider an external service when logic becomes complex, must be shared with other applications, needs specialized libraries, or must scale independently. Examples include a complex membership eligibility engine, third-party synchronization, or a reusable reporting API.

Define a small API contract: request fields, response format, authentication, timeout, retry policy, and error codes. Avoid passing unnecessary personal or medical data to the service.

Pro-code extension should solve a real limitation, not be added simply to make the architecture look more advanced. Simple age validation and date checks are understandable inside Creator. Moving them to an external service would introduce another dependency and could make ordinary form submissions less reliable.

An external API should have a documented contract: required input fields, response fields, authentication method, timeout, error codes, and versioning policy. Validate inputs on the server even if the Creator form already validates them, because an endpoint may later be called by another client that does not use the same form.

Operational responsibility increases with a pro-code extension. The team must manage deployment, secrets, logging, monitoring, retries, and service failures. Use a stable request identifier to make retries safe, and make sure the Creator workflow can distinguish a successful response from a timeout or an invalid response.

Keep the Creator app functional when an external service is temporarily unavailable, or define clearly when the operation must fail closed. The correct behaviour depends on business risk. Test service recovery and ensure retries do not duplicate records or send duplicate notifications.

Version the API contract so that a change in response fields does not silently break the Creator workflow. If the service evolves, support the old response during a transition or deploy the Creator and service changes in a coordinated sequence.

External services should return useful status codes and predictable response structures. Creator should handle an expected business rejection differently from an unexpected server failure, and user-facing messages should not expose stack traces or secret configuration.

Use an explicit timeout so a form submission does not wait indefinitely for an external service. Decide whether the Creator record is saved before the call, after the call, or in a pending state. Each choice affects the user experience and recovery procedure, so it should be documented.

Test retries with the same request identifier to prove that a repeated call does not create duplicate work. Also test an unexpected response format. The workflow should handle the failure predictably and provide a safe message instead of silently treating the call as successful.

Design / configuration summary

Need	Preferred approach
Simple field validation	Creator validation/workflow
Reusable in-app logic	Deluge custom function
Complex reusable service	Catalyst function/API or other service
Cross-app data exchange	Authenticated API connection
High reliability requirement	Timeouts, retries, idempotency, logs

3. Example Architecture and Procedure

Proposed flow: user submits valid enrollment in Creator; a workflow calls a custom function or authenticated endpoint; the service validates or processes the limited payload; the response is checked; and Creator records the outcome or displays a safe error.

First, document the required input and output. For example, the service may accept a record identifier and membership plan, then return an eligibility status and reason code. It should not need the full residential address or health notes unless those fields are essential.

If using Catalyst, create a project and select a supported function/service approach, implement the endpoint, configure environment variables and secrets securely, test it independently, then configure Creator's supported connection/API invocation. Exact console labels and runtimes may change.

Pro-code extension should solve a real limitation, not be added simply to make the architecture look more advanced. Simple age validation and date checks are understandable inside Creator. Moving them to an external service would introduce another dependency and could make ordinary form submissions less reliable.

An external API should have a documented contract: required input fields, response fields, authentication method, timeout, error codes, and versioning policy. Validate inputs on the server even if the Creator form already validates them, because an endpoint may later be called by another client that does not use the same form.

Operational responsibility increases with a pro-code extension. The team must manage deployment, secrets, logging, monitoring, retries, and service failures. Use a stable request identifier to make retries safe, and make sure the Creator workflow can distinguish a successful response from a timeout or an invalid response.

Keep the Creator app functional when an external service is temporarily unavailable, or define clearly when the operation must fail closed. The correct behaviour depends on business risk. Test service recovery and ensure retries do not duplicate records or send duplicate notifications.

Version the API contract so that a change in response fields does not silently break the Creator workflow. If the service evolves, support the old response during a transition or deploy the Creator and service changes in a coordinated sequence.

External services should return useful status codes and predictable response structures. Creator should handle an expected business rejection differently from an unexpected server failure, and user-facing messages should not expose stack traces or secret configuration.

Use an explicit timeout so a form submission does not wait indefinitely for an external service. Decide whether the Creator record is saved before the call, after the call, or in a pending state. Each choice affects the user experience and recovery procedure, so it should be documented.

Test retries with the same request identifier to prove that a repeated call does not create duplicate work. Also test an unexpected response format. The workflow should handle the failure predictably and provide a safe message instead of silently treating the call as successful.

Procedure and configuration checklist

- Define the business requirement that Creator cannot meet efficiently.
- Document the API request/response contract.
- Implement and secure the custom service or function.
- Test failure paths, retries, and end-to-end behaviour.

4. Security, Reliability and Testing

Authenticate service calls; never expose secrets in browser code or public form fields. Validate payloads on the server, use least privilege, and avoid trusting client-supplied values without checks.

Design for timeouts, transient failures, duplicate requests, and service unavailability. Use idempotency or a stable request identifier so retries do not create duplicate destination records.

Test valid input, invalid input, malformed payload, timeout, unauthorized call, duplicate request, and service recovery. Compare Creator's saved record with the external service response and preserve only useful diagnostic information.

Pro-code extension should solve a real limitation, not be added simply to make the architecture look more advanced. Simple age validation and date checks are understandable inside Creator. Moving them to an external service would introduce another dependency and could make ordinary form submissions less reliable.

An external API should have a documented contract: required input fields, response fields, authentication method, timeout, error codes, and versioning policy. Validate inputs on the server even if the Creator form already validates them, because an endpoint may later be called by another client that does not use the same form.

Operational responsibility increases with a pro-code extension. The team must manage deployment, secrets, logging, monitoring, retries, and service failures. Use a stable request identifier to make retries safe, and make sure the Creator workflow can distinguish a successful response from a timeout or an invalid response.

Keep the Creator app functional when an external service is temporarily unavailable, or define clearly when the operation must fail closed. The correct behaviour depends on business risk. Test service recovery and ensure retries do not duplicate records or send duplicate notifications.

Version the API contract so that a change in response fields does not silently break the Creator workflow. If the service evolves, support the old response during a transition or deploy the Creator and service changes in a coordinated sequence.

External services should return useful status codes and predictable response structures. Creator should handle an expected business rejection differently from an unexpected server failure, and user-facing messages should not expose stack traces or secret configuration.

Use an explicit timeout so a form submission does not wait indefinitely for an external service. Decide whether the Creator record is saved before the call, after the call, or in a pending state. Each choice affects the user experience and recovery procedure, so it should be documented.

Test retries with the same request identifier to prove that a repeated call does not create duplicate work. Also test an unexpected response format. The workflow should handle the failure predictably and provide a safe message instead of silently treating the call as successful.

Verification checklist

- Test valid, malformed, unauthorized and duplicate calls.
- Simulate timeout and service recovery.
- Verify no duplicate record or unsafe error leakage.

5. Results and Conclusion

Custom functions are a low-friction extension for reusable business logic within Creator. Catalyst or another pro-code service can provide a separate execution boundary for specialized processing, but it also increases operational responsibility.

The current screenshots show Deluge workflows but no Catalyst deployment. The experiment therefore documents the extension architecture and validation steps, not a completed external service.

Conclusion: extend the app only when the requirement justifies the added complexity. Keep routine rules in Creator, isolate complex logic behind a documented API, secure the connection, and test failure paths as carefully as success paths.

Pro-code extension should solve a real limitation, not be added simply to make the architecture look more advanced. Simple age validation and date checks are understandable inside Creator. Moving them to an external service would introduce another dependency and could make ordinary form submissions less reliable.

An external API should have a documented contract: required input fields, response fields, authentication method, timeout, error codes, and versioning policy. Validate inputs on the server even if the Creator form already validates them, because an endpoint may later be called by another client that does not use the same form.

Operational responsibility increases with a pro-code extension. The team must manage deployment, secrets, logging, monitoring, retries, and service failures. Use a stable request identifier to make retries safe, and make sure the Creator workflow can distinguish a successful response from a timeout or an invalid response.

Keep the Creator app functional when an external service is temporarily unavailable, or define clearly when the operation must fail closed. The correct behaviour depends on business risk. Test service recovery and ensure retries do not duplicate records or send duplicate notifications.

A good conclusion separates observed results from expected results. Report any failed test and the next corrective action plainly; a documented limitation is more useful than claiming a pass that the evidence does not support.

Learning outcomes

- Distinguished Deluge from external pro-code services.
- Defined an API contract and security requirements.
- Designed failure and recovery tests.

Conclusion

Conclusion: extend the app only when the requirement justifies the added complexity. Keep routine rules in Creator, isolate complex logic behind a documented API, secure the connection, and test failure paths as carefully as success paths.

References

Zoho Catalyst: https://catalyst.zoho.com/

Zoho Deluge: https://www.zoho.com/deluge/help/

Zoho Creator API: https://www.zoho.com/creator/help/api/

2414011012

6. API Contract and Extension Lifecycle

An API contract should define endpoint purpose, request fields, response fields, authentication, timeout, expected error codes, and versioning. Keep the contract small so the Creator workflow is not tightly coupled to internal service implementation.

Keep secrets in supported connection or environment-secret facilities rather than source code. Validate inputs on the server, restrict outbound network permissions where possible, and return generic user-facing errors while retaining safe diagnostic detail for administrators.

Treat the external function as a separate software component: version it, test it independently, deploy it through a repeatable process, and monitor failures. A pro-code extension is not maintenance-free; it trades some visual simplicity for greater control and responsibility.

Version the API contract so that a change in response fields does not silently break the Creator workflow. If the service evolves, support the old response during a transition or deploy the Creator and service changes in a coordinated sequence.

External services should return useful status codes and predictable response structures. Creator should handle an expected business rejection differently from an unexpected server failure, and user-facing messages should not expose stack traces or secret configuration.

Practical application to the lab

The Gym Enrollment app already uses Deluge workflows for age and start-date validation and for email confirmation. A pro-code extension could provide more complex integrations, reusable services, or advanced processing. No Catalyst service is shown in the supplied screenshots, so this report presents an extension design and procedure rather than claiming that a Catalyst deployment exists.

Evaluation criteria

A satisfactory lab result should be reproducible, supported by a clear configuration or test record, and honest about limitations. Record the setup, test input, expected result, actual result, and evidence. If a feature was not enabled or tested in the current account, report that as a limitation and describe the procedure needed to complete it.

Checklist for submission

- Confirm the report matches the experiment title and stated objectives.
- Keep screenshots and configuration details aligned with what was actually performed.
- Separate verified outcomes from proposed implementation steps.
- Recheck the final PDF page layout before submission.

7. Extension Test Plan and Final Recommendations

Test the external service separately and then test the end-to-end Creator workflow. Include malformed payloads, authorization failures, timeouts, duplicate submissions, and recovery after the service becomes available again.

Monitor service latency and failure rates. Ensure the user receives a useful outcome even when the external service is temporarily unavailable, and do not leave the enrollment record in an ambiguous state.

Final recommendation: keep simple business rules in Creator, use custom functions for reusable in-platform logic, and introduce Catalyst only when a clear requirement justifies an external service.

Version the API contract so that a change in response fields does not silently break the Creator workflow. If the service evolves, support the old response during a transition or deploy the Creator and service changes in a coordinated sequence.

External services should return useful status codes and predictable response structures. Creator should handle an expected business rejection differently from an unexpected server failure, and user-facing messages should not expose stack traces or secret configuration.

Practical application to the lab

The Gym Enrollment app already uses Deluge workflows for age and start-date validation and for email confirmation. A pro-code extension could provide more complex integrations, reusable services, or advanced processing. No Catalyst service is shown in the supplied screenshots, so this report presents an extension design and procedure rather than claiming that a Catalyst deployment exists.

Evaluation criteria

A satisfactory lab result should be reproducible, supported by a clear configuration or test record, and honest about limitations. Record the setup, test input, expected result, actual result, and evidence. If a feature was not enabled or tested in the current account, report that as a limitation and describe the procedure needed to complete it.

Checklist for submission

- Confirm the report matches the experiment title and stated objectives.
- Keep screenshots and configuration details aligned with what was actually performed.
- Separate verified outcomes from proposed implementation steps.
- Recheck the final PDF page layout before submission.

24140_012